The score and the scorecard as they are today

How today's score is computed and how today's scorecard judges screeners, step by step, and why neither should be copied as is. The intent for their replacements is in IDEAS.md (parts 1 and 3); this document exists so the rebuild knows exactly what it is replacing.

Written from `src/scoring/processor.py`, `scorer.py`, `server.py`, `src/sideE/score.js`, `src/training/trainingData.js`, `src/scheduler.js` and `src/analysis/screenerReport.js`.

> **Status:** the scorer is switched off on every tool (D-D1), so no card has a score today.

The moving parts

Part	Where	Does
Training rows	`r4a_train`, `r4b_train` in each tool's database	one row per card per day: the R1 photo joined with its R3 outcome (REGISTERS.md)
Export	`src/training/trainingData.js`	writes the rows as CSV to the tool's temp folder for Python
Trainer	`src/scoring/processor.py`	builds the tables (below) into `src/scoring/outputs[-tN]/`
Live scorer	`src/scoring/scorer.py`, served by `server.py` (Flask) as `scorer-Tn` on the tool's port + 1	scores one card
Caller	`src/sideE/score.js`	at discovery, sends the card with its bias and entry time; stores `_score` and `_scoreDetails`
Schedule	`Scorer Auto-Train 4:20 PM`	retrains on all history after the outcomes are in; skipped when rows are missing

---

Training (`processor.py`)

Six bases

Base	Rows	Learns
B1	R4A (entry A, 09:37)	`upR_A`: how far it ran up
B2	R4A	`downR_A`: how far it ran down
B3	R4A	the larger of the two
B4	R4B (entry B, 09:40)	`upR_B`
B5	R4B	`downR_B`
B6	R4B	the larger of the two

Each base gets a **main** table from all rows, and one **sub** table per market regime from only that regime's rows.

Building one table

1. **Features.** About 60 columns from the photo:
   - **categorical**, one-hot encoded: sector, industry, regime, sector bias, themes, catalyst, the screener keys, the three trend stages, the bias, and the unit-free relations (above or below each MA, VWAP, previous close, open, PM high and low; the MA stack; the month quarter; the PM/ADR band);
   - **numeric**, standardised: price, change, gap, VWAP, the MAs, RVOL, ATR, ADR %, day and month high/low, range positions, cap, float, short %, PM range and PM/ADR, sector score, minutes from the open at discovery, the % distance to each level.
2. **PCA.** Keep the factors whose eigenvalue is above 1 (at least one).
3. **Buckets.** Each factor's scores are cut into **5** equal-count buckets (`SCORER_N_BUCKETS`).
4. **Bucket score.** For each bucket, from the outcome of its rows:
   - mean R, and win rate = share of rows with R **> 1.3**;
   - each normalised by the best bucket of that factor;
   - raw = (0.5 × mean_norm + 0.5 × winrate_norm) × 100;
   - confidence = n ÷ (n + 5);
   - **FinalScore = raw × confidence.**
5. **Saved:** the factor importance table, one bucket CSV per factor, and `metadata.pkl` (encoder, scaler, PCA, bucket edges). Tables not rebuilt by this run are deleted, so an old regime cannot keep scoring cards.

Scoring a card (`scorer.py`)

1. **Base.** From the bias and the entry time (`scorerEntryTime`): Long → upR (B4, or B1 at 09:37), Short → downR (B5 / B2), no bias → the larger (B6 / B3).
2. **Table.** The card's regime sub table, if it exists and has at least `regimeSampleThreshold` rows (150); otherwise main.
3. **Encode** the card exactly as in training and project it onto the factors.
4. **Look up** the bucket each factor score falls in (clamped to the ends) and take its FinalScore.
5. **Score = the plain average** of those bucket scores, 0–100.

The breakdown (`_scoreDetails`) carries the base, the table, its sample count, the confidence, and each factor's bucket score: what the card's score popup shows (CARD.md).

Why it should not be copied

Problem	Effect
Too many features for the rows. About 60 columns, one-hot industries and screener keys, against a few hundred rows per tool, and far fewer per regime	the factors describe noise; a sub table rests on a handful of days
Factors averaged equally. A factor explaining 30 % of the variance counts the same as one explaining 3 %	the score is pulled to the middle
Relative, not absolute. Each bucket is normalised by the best bucket of its factor	in a table where nothing moves, the best of the bad buckets still scores near 100
The good-move line differs. Here a win is R **> 1.3**; the scorecard and the ideas say **≥ 1.3**	the score and the judge grade on different curves
PCA is not explainable to a trader. "Factor 3, bucket 4" says nothing about the card	the popup cannot say why
No minimum sample. A table built from a few rows still scores	thin scores look as sure as thick ones
Leftover parts. `server.py` still serves per-setup grading routes (`setupProcessor.py`, `setupScorer.py`) built for the removed trading module; they read a `desk.db` that no longer exists	dead code beside live code

What is worth keeping: training only on the tool's own frozen photo, outcomes only from after the entry, retraining each day after the close, the confidence that grows with the sample, and carrying the breakdown with every score. All of these are rules in IDEAS.md.

Judging screeners today (`src/analysis/screenerReport.js`)

The **Screener scorecard** on the Analysis tab, and through `/api/analysis/screener-report?entry=A|B` the comparison page (PAGES.md).

How it is computed

1. Read every training row of the chosen entry (`r4a_train` for A, `r4b_train` for B): that is every captured day, plus any uploaded rows.
2. For each row, take `upR` (the move up from the entry, in ATR) and `downR`, and credit the row to **every screener in its** `screenerKeys`: a card found by two screeners counts for both.
3. Per screener:

Number	Formula
cards, days	rows and distinct dates
good %	share of rows with `upR` **≥ 1.3**
big %	share with `upR` ≥ 2
median and average R	of `upR`
average −R	of `downR`
days hit (consistency)	share of days whose **best** card reached 1.3

4. **Verdict:** under 20 cards or under 5 days is “not enough data”; otherwise **keep** when good % ≥ 35 and days hit ≥ 50; **watch** when good % ≥ 25 or days hit ≥ 40; **drop** otherwise.
5. **Baseline:** all rows of the tool pooled, the comparison that counts, because one tool samples $2 small caps and another large caps.
6. Judged screeners are listed first, then by good %; mirror pairs are recorded from `mirrorOf`.

Why it should not be copied as is

Problem	Effect
Every screener is graded on the move up (`upR`), including bearish ones (most mirrors) (D-S12)	a short setup that worked reads as a failure, and the mirror comparison is meaningless
The model calls a win **> 1.3**, the scorecard **≥ 1.3**	two curves
The pooled baseline counts a card once per screener that found it	the baseline leans towards cards several screeners agree on
Uploaded external rows are mixed with captured ones	a verdict can rest on data the tool never collected
The thresholds (35 / 50, 25 / 40, 20 cards, 5 days) are fixed in code	they are shown on the page but cannot be tuned

What is worth keeping: measuring the move, not a trade; consistency across days as the deciding number; “not enough data” instead of a verdict; the tool’s own baseline; the benchmark and the mirror comparisons. All are rules in IDEAS.md, part 3.